



BASE DE DONNÉES

Module 2

RESUME

Synthèse

Charlotte Counson

[Titre du cours]

Table des matières

Chapitre 1 – Généralités.....	3
PL/SQL	3
Définition de blocs de programmation SQL	3
Procédures stockées en PL/SQL	3
Chapitre 2 – Les types de données et les variables.....	3
Types de données scalaires	3
Types de données LOBS	4
Structure d'un bloc.....	5
Déclaration de variables	5
Types composites ou composés.....	5
Conversion de types	6
Type REF	6
Visibilité des variables	6
Opérateurs et expressions	6
Logique trivalente et valeur NULL.....	6
Séquence et pseudo-colonnes du PL/SQL.....	6
Chapitre 3 – Les structures de contrôle	7
Les structures conditionnelles.....	7
Les structures itératives	7
Chapitre 4 – Les exceptions	7
Utilisation des exceptions	7
Catégories d'exceptions	7
Chapitre 5 – Les collections.....	8
Définir des types de collections.....	8
Les méthodes associées aux collections	8
Chapitre 6 - Des records au collection bulk.....	9
Définitions	9
Opérateurs	9
Chapitre 7 – Les procédures et les fonctions	10
CREATE PROCEDURE	10
Compilation	10
Exécuter une procédure	10
Création d'une fonction	11
Exécuter une fonction	11
Utiliser les paramètres avec NOCOPY	11

RAISE_APPLICATION_ERROR.....	11
Les notations nommées	11
Les paramètres par défaut	11
Chapitre 8 - Les packages	11
Le concept du package	11
Persistance de la session	12
Principaux avantages des packages	12
Procédures et fonction polymorphes.....	12
Chapitre 9 – Les curseurs	12
Curseurs explicites.....	12
Curseurs for implicites.....	13
Curseurs for explicites	13
Curseurs avec paramètres.....	13
Curseurs for update.....	13
Le curseur implicites SQL.....	14
Chapitre 10 – Les déclencheurs	14
Déclencheurs réagissant aux instructions LMD	14
Contourner les tables mutantes.....	14
Déclencheurs réagissant aux instructions du LDD	15
Déclencheurs réagissant aux événements système.....	15

PL-SQL

Chapitre 1 – Généralités

PL/SQL : Procedural Language extension to SQL, c'est un langage procédural qui permet de traiter de manière structurée les données retournées par une instruction SQL.

C'est un langage séquentiel en short circuit (permet de limiter les dégâts en évitant les dégâts)

PL/SQL n'est pas une norme à la différence de SQL qui en est une.

PL/SQL n'a aucun aspect normatif contrairement à SQL. PL/SQL permet de stocker du code sur le serveur et sert principalement à programmer des procédures stockées et déclencheurs.

PL/SQL

Meilleure perf en écrivant des blocs PL/SQL :

- Les ordres SQL sont transmis par bloc au moteur de la BD
- Le moteur PL/SQL est optimisé ce qui améliore encore les perfs des applications.

Une autre force sont des différents types de données manipulés

Définition de blocs de programmation SQL

- Les portions de code PL/SQL peuvent être définies dans n'importe quelle interface SQL.
- Il suffit d'encoder la portion de code comme s'il s'agissait d'une instruction SQL et l'exécuter
- Dans cette portion, il est possible de réaliser des affichages, d'intercepter des exceptions et de les traiter

Procédures stockées en PL/SQL

- On obtient de meilleure performance et moins de trafic réseau ainsi qu'un meilleur temps de réponse
- Ça se trouve dans la mémoire cache de la BD et peuvent être partagées entre les multiples utilisateurs
- L'indépendance données-programmes s'en trouve renforcée !

Architecture PL/SQL

- Le SQL et PL/SQL comportent chacun un moteur d'exécution associé.
- Quand le serveur reçoit un appel, la version compilée du programme est chargée
- Le moteur PL/SQL gère les structures en mémoire et le flux logique du programme

Chapitre 2 – Les types de données et les variables.

Types de données scalaires

Les quatre catégories de données :

- **Scalaires** : Ces types sont atomiques : ce sont les types utilisés pour définir une colonne d'une table
- **Composés** : Ces types comprennent plus d'un élément ou composant.
- **Références** : Ces types permettent de définir des références vers d'autres types
- **Grands objets** : Ces types de données spécifient la localisation d'un « grand objet » binaire comme une image stockée dans la base de données ou un fichier externe.

Les quatre types de données scalaires :

➤ Types de données numériques

- NUMBER[(P,S)]
 - P : nb de chiffres significatifs
 - S : nb décimaux
 - NUMBER possède quelques sous-types permettant une compatibilité ANSI/ISO .
 - Génère une exception
- BINARY_INTEGER
 - Nombre entier signé
 - C'est un vieux truc, problème de compatibilité
- PLS_INTEGER
 - Plus performant que les 2 autres. Même s'ils ont le même rang, ils n'ont pas le même comportement.
 - Génère des exceptions.
- BINARY_FLOAT et BINARY_DOUBLE
 - Sont là depuis Oracle 10.
 - Permet d'augmenter la perf dans les app

Syntaxe

```
1 TYPE type_name
2 IS TABLE OF element_type [NOT NULL]
3 INDEX BY [PLS_INTEGER |
4         BINARY_INTEGER |
5         VARCHAR2 (size_limit)];
```

➤ Types de données « caractères »

- CHAR
 - Chaines de longueur fixe
 - Stockage dépendant du jeu de caractère
 - Taille exprimée par défaut en bytes.
- VARCHAR2
 - Chaines de longueur variable
 - Stockage dépendant du jeu de caractères de DB
 - Taille exprimée par défaut en bytes
- LONG
 - Comme le type VARCHAR2
 - LONG RAW n'est pas interprété par le PL/SQL
- RAW
 - Ce type de données permet de traiter des données binaires ou des char en bin
- ROWID
 - C'est une colonne que l'on donne pour chaque table qui contient des valeurs binaires. Ces valeurs représentent l'adresse de stockage de la ligne correspondante.
 - On peut caster ses rowid (Select cast (rowid as char(30))
- NChar

➤ Types de données booléen

➤ Types de données DATE, TIME, INTERVAL

- Type DATE
- Type TIMESTAMPS
- Type INTERVAL DAY [(précision)] to second [(précision)]
- Type INTERVAL YEAR [(précision)] to month

Types de données LOBS

Ce sont regroupés sous cette appellation tous les large objects

Ils peuvent contenir des données non structurées pouvant aller jusqu'à 4 GB.
Nous ne les aborderons pas dans le cours.

Structure d'un bloc

Une section de déclaration de variables

Une section d'instructions

Une section d'exception

Déclaration de variables

Déclaration de variables

```
1 Nom_Variable [CONSTANT] TYPE [NOT NULL]
2 [{DEFAULT | := ] VALEUR};
```

- Nom_variable : ce nom de variable doit être unique dans le bloc
- TYPE : le type de la variable qui peut être un type scalaire ou un type composite
- CONSTANT : La valeur de la variable est une constante qui ne sera pas modifiable dans le bloc
- NOT NULL : la variable doit obligatoirement être assignée, sinon une erreur est générée
- := VALEUR : la variable est initialisée avec la valeur

Types composites ou composés

Ces types de données font partie des types de données définis par le programmeur PL/SQL.

Parmi ces types, on retrouve également les types dérivés ou sous-types, objets de la section suivante.

- Type RECORD
 - Permet de déclarer des types de var composites
 - C'est comme un enregistrement ou une structure de données d'un langage de 3ieme génération
 - Il doit faire l'objet d'une déclarations d'un type de données et ensuite une variable peut être définie à l'aide de ce type
 - Pour déclarer un RECORD, on doit faire :
 - Déclarer un TYPE correspondant à la structure voulue
 - Utiliser ce TYPE pour déclarer une variable
- Utilisation de %TYPE
 - L'attribut permet de déclarer une variable dont le type est basé sur le type d'une colonne ou d'une autre variable.
 - Les variable déclarée en %TYPE héritent du type de données de la colonne.
- Utilisation de %ROWTYPE
 - Il n'est pas nécessaire de connaître le nom de toutes les colonnes d'une table ni de l'ordre des colonnes.

Utilisation de %ROWTYPE

```
1 DECLARE
2     UnEmploye Emp%ROWTYPE;
3 BEGIN
4     SELECT * INTO UnEmploye FROM emp WHERE empno = 7788;
5     DBMS_OUTPUT.PUT_LINE('Le client 7788 : ' ||
6                           UnEmploye.Ename || ' ' ||
7                           UnEmploye.Job);
8 END;
```

Conversion de types

- Conversion explicites
Fonction normalisée CAST dispo depuis Oracle 9iR2

Type REF

Les types REF représente des valeurs correspondant à des pointeurs qui permettent de référencer d'autre objet

Certaines de ces utilisations relèvent des extensions objet-relationnelles.

On n'aborde pas ça.

Visibilité des variables

Une variable est référencée par son nom et est visible dans le bloc PL/SQL où elle a été déclarée.

Elle est visible également dans tous les blocs imbriqués dans celui où la déclaration à été faite pour autant qu'elle n'ait pas été redéfinie.

Opérateurs et expressions

Les expressions PL/SQL sont construites en appliquant des opérateurs à des opérandes

Une opérande peut être une variable, une constante.

Les expressions sont évalué suivant la préséance des opérateurs.

Opérateur	Opération
**	Exponentiation
-	Opposé
*, /	Multiplication, division
+, -,	Addition, soustraction concaténation
=, <, >, <=, >=, <>, !=, ~=, ^=, IS NULL, LIKE, BETWEEN, IN	Comparaison
NOT	Négation logique
AND	Conjonction (ET Logique)
OR	OU Logique

Logique trivalente et valeur NULL

- Vrai
- Faux
- Inconnu
- NVL
- NULLIF
- NVL2

Séquence et pseudo-colonnes du PL/SQL

Le PL/SQL reconnaît les pseudo-colonnes de SQL telles que ROWID, CURRVAL, NEXTVAL, LEVEL et ROWNUM.

- ROWNUM permet de spécifier l'ordre dans lequel une ligne est sélectionnée dans une table
- ROWNUM2 et ainsi de suite
- ROWNUM est attribué avant l'opération de tri (order by)

Chapitre 3 – Les structures de contrôle

Les structures conditionnelles

- IF THEN
- IF THEN ELSE
- CASE

Les structures itératives

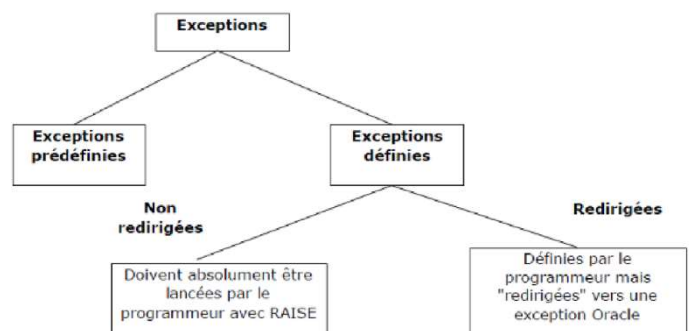
- WHILE
- FOR

Chapitre 4 – Les exceptions

Utilisation des exceptions

- Déclaration des exceptions
 - `Nom_de_l_exception EXCEPTION;`
- Le gestionnaire d'exceptions
 - Est chargé d'intercepter les erreurs
 - On peut déclarer partout dans le bloc
 - Elles sont lancées explicitement avec RAISE et implicitement par Oracle
- La clause WHEN OTHERS
 - Permet d'intercepter n'importe quelle exception non gérée

Catégories d'exceptions



- Exceptions définies « non redirigées »
 - Doivent être déclarées
 - Doivent être lancées par le programmeur avec la commande RAISE
 - Ne sont pas associées à une erreur prédéfinie par Oracle
 - Utile pour gérer une exception spécifique liée à la logique de prog du bloc
- Exceptions prédéfinies : NO_DATA_FOUND
 - NO_DATA_FOUND ORA-01403 +100**
- Exceptions prédéfinies : TOO_MANY_ROWS
 - TOO_MANY_ROWS ORA-01422 -1422**
- Exceptions prédéfinies : DUP_VAL_ON_INDEX
 - DUP_VAL_ON_INDEX ORA-00001 -1**
- Exceptions prédéfinies : INVALID_NUMBER
 - INVALID_NUMBER ORA-01722 -1722**
- Exceptions prédéfinies : VALUE_ERROR
 - **VALUE_ERROR ORA-06502 -6502**
- Exceptions définies « redirigées »

- Fait par le prog et associé à un code d'erreur
- Pas besoin de RAISE
- Se définit en utilisant le pragma EXCEPTION_INIT


```
PRAGMA EXCEPTION_INIT (NameExc, Oracle_error);
```
- On ne lance pas explicitement l'exception
- Exceptions définies « redirigées » : intégrité référentielle
- Exceptions définies « redirigées » : Ressource Busy
 - Les exceptions remontent de bloc en bloc jusqu'au code appelant si elles ne sont pas capturées.

```
EXCEPTION
WHEN ExcRessourceBusy THEN
DBMS_OUTPUT.PUT_LINE('La ressource est occupée')
```
- Clause WHEN OTHERS
 - Doit être contenu dans le bloc
 - Erreurs pendant les opés arithmétiques, opés de BD ainsi que de conversion de données surviennent souvent.
 - Pour chaque exception interceptée on doit se demander si les opé faite doivent être annulé ou confirmé
 - Les exceptions peuvent faire partie intégrante de la logique de prog

Chapitre 5 – Les collections

Définir des types de collections

Une collection est un ensemble, éventuellement ordonné, d'éléments de même type. Chaque élément est repéré au moyen d'un indice.

Le PL/SQL possède 3 types de collections :

- Les tableaux associatifs
 - Ensemble ordonné d'éléments repéré par un indice de type numérique au char. On parle également de table de hachage out table PL/SQL
- Les tables imbriquées
 - Qui peuvent contenir un ensemble non ordonné d'élément indicés par des valeurs numérique consécutives.
- Les tableaux pré dimensionnés
 - Ensemble ordonné d'éléments de même type dont le nb d'éléments est fixé lors de la déclaration. Les éléments sont indicés par des nb consécutifs.

Les méthodes associées aux collections

- Déclaration
 - Il n'y a pas de clause d'initialisation ni de constructeur associé à un type tableau associatif !

➤ Initialisation

```

1 declare
2   ConstHireDate    CONSTANT NUMBER := -20001;
3   ConstNom         CONSTANT NUMBER := -20002;
4   ConstHireDateNull CONSTANT NUMBER := -20007;
5   TYPE TypeTableMessErreur
6     IS TABLE OF VARCHAR2(200)
7     INDEX BY BINARY_INTEGER;
8   TableMessErreur  TypeTableMessErreur;
9 BEGIN
10  --initialisation
11  TableMessErreur(ConstHireDate) :=
12    'Date embauche > date du jour';
13  TableMessErreur(ConstNom) := 'Nom employé inconnu';
14  TableMessErreur(ConstHireDateNull) :=
15    'Date embauche inconnue';
16 END;

```

```

1 DECLARE
2   TYPE TypeLesEmployes
3     IS TABLE OF Emp%ROWTYPE
4     INDEX BY BINARY_INTEGER;
5   TableEmployes  TypeLesEmployes;
6 BEGIN
7   -- On charge dans le tableau le résultat d'une
8   requête !
9   SELECT * BULK COLLECT
10    INTO TableEmployes
11    FROM Emp;
12 EXCEPTION
13   WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE(SQLERRM);
14 END;

```

➤ Méthode exists

```

SELECT * BULK COLLECT INTO LesEmployes FROM Employes;
IF LesEmployes.EXISTS(25) -- Test sur l'indice

```

➤ Méthode count

```

DBMS_OUTPUT.PUT_LINE(LesEmployes.COUNT);
SELECT * BULK COLLECT INTO LesEmployes FROM Employes;

```

➤ Méthode first et last

- Déterminé le premier et le dernier indice des éléments. Si la collection est vide ca First et LAST donne NULL.

```

FOR i IN LesEmployes.FIRST..LesEmployes.LAST
  LOOP

```

➤ Méthode next

```

BEGIN
  i := LesEmployes.FIRST;
  WHILE i IS NOT NULL LOOP
    DBMS_OUTPUT.PUT_LINE(i || ' : ' || LesEmployes(i));
    i := LesEmployes.NEXT(i); -- Ici
  END LOOP;

```

➤ Méthode delete

- DELETE supprime tous les éléments d'une collect
- DELETE(n) supprime le nième élément d'une table
- DELETE(m,n) supprime tous les éléments dans la fourchette m..n

Exceptions liées aux collections

- NO_DATA_FOUND : Un indice désigne un élément supprimé ou un élément qui n'existe pas dans la table.
- VALUE_ERROR : Un indice est null ou ne peut être converti dans le type d'indice.
- COLLECTION_IS_NULL : On utilise une collection atomically null
- SUBSCRIPT_BEYOND_COUNT : Un indice dépasse le nombre d'éléments de la collection.
- SUBSCRIPT_OUTSIDE_LIMIT : Un indice est en dehors de la fourchette permise.

Chapitre 6 - Des records au collection bulk

Définitions

- Les records permettent de définir des structures de données hétérogènes

Opérateurs

- Opérateur d'accession
 - « . » opérateur de priorité
- Comparaison

- Pour comparer deux records, il faut les comparer champ par champ. L'idéal est donc d'écrire une fonction qui prend 2 records en paramètres et renvoie vrai ou faux
- « = »
- Affectation multiple
- RETURNING INTO
 - L'utilisation de la clause RETURNING INTO dans les commandes, insert, update et delete évite de refaire une nouvelle lecture pour connaître les nouvelles valeurs d'une ligne !
- BULK COLLECT INTO
 - NO_DATA_FOUND n'est pas déclenché lorsque le résultat de la recherche est vide !

```
SELECT * BULK COLLECT INTO LesEmployes FROM Emp
WHERE Deptno = 10;
```
- FORALL
 - Permet d'envoyer 1 fois un lot d'instructions
 - Pour accélérer les opé de base, on les met dans un forall. Pour optimiser les instructions on utilise la clause INTO BULK COLLECT
 - Peut déclencher une exception non gérée, toutes les modifs réalisées par les instructions précédentes seront annulées
- SQL%BULK_ROWCOUNT
 - Le nombre de lignes impliquées dans chaque instruction LMD d'un FORALL peut être déterminé au moyen de l'attribut composé
- Save exceptions
 - Mécanisme d'exception déclenché pendant le FORALL
 - Déclenché ce mécanisme pendant le SAVE EXCEPTIONS permet de sauver les infos de l'exception et continuer le traitement
 - L'absence de la clause dans le FORALL provoque l'arrêt de l'instruction dès la première exception rencontrée

Chapitre 7 – Les procédures et les fonctions

CREATE PROCEDURE

- Une procédure ou une fonction peut également être déclarée localement dans une autre

Création d'une procédure

```
CREATE PROCEDURE NomDeLaProcédure [(,.,)] IS
-- déclaration des variables locales ou types ou
-- curseurs
BEGIN
...
EXCEPTION
...
END;
```

Compilation

- Lors de la compilation d'un objet, le moteur PL/SQL génère les messages d'erreurs

Exécuter une procédure

Appeler une procédure

```
SET SERVEROUTPUT ON
BEGIN
Afficher('11111111'); -- ici
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('ERREUR : ' || SQLCODE ||
SQLERRM);
END;
```

Création d'une fonction

Sortir d'une fonction

```
1 CREATE FUNCTION NomDeLaFonction [(,.,)]
2 RETURN return_type IS
3 [déclaration des variables locales ou types ou curseurs]
4 BEGIN
5 RETURN ... ; -- sortie
6 EXCEPTION
7 ...
8 RETURN ... ; -- sortie
9 END
```

Exécuter une fonction

```
1 BEGIN
2 UnEmploye := Rechercher('121212');
3 DBMS_OUTPUT.PUT_LINE('Nom : ' || UnEmploye.Nom);
```

Utiliser les paramètres avec NOCOPY

- Les paramètres OUT et IN sont passés par valeur
- Pendant l'exécution, des var temp sont créés pour contenir des valeurs des paramètres de type OUT
- Si le sous-programme termine normalement son exécution, les valeurs sont alors recopiées dans les paramètres.
- Ces copies ralentissent fortement les performances et encombrent la mémoire
- Pour éviter ces inconvénients : NOCOPY

RAISE_APPLICATION_ERROR

- Compris entre -20000 et 20999

Les notations nommées

- C'est nécessaire pour utiliser les valeurs par défaut des paramètres
- Nécessaire pour spécifier les paramètres dans n'importe quel ordre

Les paramètres par défaut

- Lorsque les paramètres sont spécifiés dans le mode IN il est possible de leur affecter des valeurs par défaut

Chapitre 8 - Les packages

Le concept du package

- C'est composé de deux parties de codes distinct : la spécification et le corps
 - Spécification contient les prototypes des procédures/fonctions/déclaire variable public
 - Corps contient l'implémentation des éléments définis dans la spécification mais aussi contenir et définir des éléments privés. Également une section d'instruction appelée initialization section
- Le code défini publiquement est accessible à tout utilisateur possédant le privilège EXECUTE sur le package
- La sécurité repose sur des GRANT EXECUTE. Aucun accès aux données n'est accordé
- Le code privé est inaccessible : ceci renforce encore l'indépendance données-programmes et la réutilisabilité

```

1 CREATE OR REPLACE PACKAGE PublicVarPersistante IS
2     vPublicVar CHAR := 'Y';
3 END PublicVarPersistante;

```

Persistence de la session

- On peut établir une session
- On peut exécuter un programme qui assigne une valeur à une variable d'un package
- Cette variable est donc initialisée et persistera jusqu'à la fin de la session
- Les variables publiques sont locales à une session

Principaux avantages des packages

- Permet de regrouper plusieurs unités de programmation dans un même container réduction du nombre d'objets stocké à maintenir
- Quand on sépare l'implémentation de l'interface permet de maintenir ou modifier l'implémentation sans toucher au interface ni au programme appelants
- Il n'est même pas nécessaire de recompiler les programmes appelants si l'interface n'est pas elle-même recompilée

Procédures et fonction polymorphes

Spécification du package contenant 2 fonctions polymorphes

```

1 CREATE OR REPLACE PACKAGE Pkgclients IS
2     -- ici refClient
3     FUNCTION RechercheClient(p_refClient IN clients.
4         refClient%TYPE)
5         RETURN clients%ROWTYPE;
6     -- ici nomClient
7     FUNCTION RechercheClient(p_nomClient IN clients.
8         nomClient%TYPE)
9         RETURN clients%ROWTYPE;
10 END PkgClients;

```

Chapitre 9 – Les curseurs

Les curseurs permettent de traiter les requêtes de recherche dont le résultat contient un nombre X de tuples

Le curseur réalise le lien entre la philosophie ensembliste et l'enregistrement par l'enregistrement des langages.

C'est considéré comme un pointeur, il est toujours associé à une expression de sélection.

L'expression de la sélection est évaluée lorsque le curseur est ouvert, jusqu'au résultat du curseur et que ça se ferme.

Curseurs explicites

- Déclaration


```
CURSOR nom_curseur IS expression_de_sélection;
```
- Aucune restriction au niveau de la requête de sélection
- Un curseur DOIT ETRE déclaré avant d'être utilisé. Sinon exception
- 3 opé pour manipuler les curseurs : OPEN, FETCH et CLOSE
 - Open

L'effet du **OPEN** est d'évaluer l'expression de sélection associée.

Le curseur est dans l'état ouvert et est positionné avant la première ligne du résultat

Un curseur ouvert peut être parcouru par une série de **fetch**

La tentative d'ouverture d'un curseur déjà ouvert provoque l'exception `cursor_already_open`

- **Fetch**

Les variables de la liste ou les champs du record doivent être compatibles avec la clause `select`

Idem si on utilise **BULK COLLECT INTO**

- **Close**

La fermeture d'un curseur non ouvert déclenche l'exception `invalid_cursor`

➤ **Attribut**

		%FOUND	%NOTFOUND	%ISOPEN	%ROWCOUNT
OPEN	Avant	Exception	Exception	FALSE	Exception
	Après	NULL	NULL	TRUE	0
1er FETCH	Avant	NULL	NULL	TRUE	0
	Après	TRUE	FALSE	TRUE	1
Nème FETCH	Avant	TRUE	FALSE	TRUE	*
	Après	TRUE	FALSE	TRUE	*
Dernier FETCH	Avant	TRUE	FALSE	TRUE	*
	Après	FALSE	TRUE	TRUE	*
CLOSE	Avant	FALSE	TRUE	TRUE	*
	Après	Exception	Exception	FALSE	Exception

Curseurs for implicites

Du point de vue programmation : ce sont les plus simples ! Il suffit de placer la requête de sélection.

```
BEGIN
  FOR Employe IN
    (SELECT *
     FROM Emp
     WHERE job LIKE 'A%')
  LOOP
    DBMS_OUTPUT.PUT_LINE(Employe.nom);
  END LOOP;
END;
```

Curseurs for explicites

- Déclaration explicite
- Son parcours est réalisé au moyen d'une boucle `for` et d'un record adhoc implicitement déclaré dans la boucle
- Il est possible d'utiliser les attributs `%FOUND`, `%NOTFOUND`, `%ISOPEN`, `%ROWCOUNT`

Curseurs avec paramètres

- Il est possible de passer des paramètres à un curseur
- La déclaration des paramètres dans le curseur se fait lors de la déclaration du curseur, de la même manière que pour une procédure
- L'évaluation et la transmission des paramètres se fait au moment de l'ouverture du curseur

Curseurs for update

- Les curseurs `for update` permettent de modifier le tuple qui vient d'être extrait par un `FETCH`
- Oracle place des verrous exclusifs sur les tuples de l'active set des curseurs `for update`

```
SELECT ... FROM ...
FOR UPDATE [OF liste_colonnes]
[WAIT/NOWAIT]
```

Le curseur implicites SQL

- Ne nécessite pas de déclaration
- Ne doit pas être ouvert, ni fermé
- Peut utiliser les attributs %FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT
- Permet d'exécuter des INSERT, UPDATE, DELETE, SELECT INTO d'une tuple

Chapitre 10 – Les déclencheurs

- Un déclencheur permet de définir un ensemble d'action qui sont déclenchées automatiquement par le SGBD lorsque certains phénomènes se produisent.
- Les actions sont enregistrées dans la base et non plus dans les programmes d'app
- Notion de SQL3
- Permet de réaliser des opés sophistiquées
- Possible être d'activer avec commandes du LDD ou à certains événements

Les déclencheurs permettent de définir des contraintes dynamiques, ils peuvent être utilisés pour :

- Générer automatiquement une valeur de clé primaire.
- Résoudre le problème des mises à jour en cascade
- Enregistrer les accès à une table
- Gérer automatiquement la redondance
- Empêcher la modification par des personnes non autorisées
- Mettre en œuvre des règles de fonctionnement plus complexes

Déclencheurs réagissant aux instructions LMD

Syntaxe

```
1 CREATE TRIGGER nom_déclencheur
2 BEFORE | AFTER
3 DELETE | INSERT | UPDATE [OF liste_colonne]
4 ON nom_table | nom_vue
5 [FOR EACH ROW]
6 [WHEN condition]
7 [bloc PL/SQL];
```

Les restrictions d'un déclencheur :

- Un déclencheur ne peut contenir de COMMIT ni de ROLLBACK
- Il est impossible d'exécuter une commande du LDD dans un déclencheur
- Un déclencheur de niveau de ligne ne peut pas :
 - Lire ou modifier le contenu d'une table mutante (table en cours de modif)
 - Lire ou modifier les colonnes d'une clé primaire, unique ou étrangère d'une table contraignante

Contourner les tables mutantes

```
1 CREATE OR REPLACE TRIGGER departementsnumdep
2 BEFORE INSERT ON Departements -- Ici
3 FOR EACH ROW
4 BEGIN
5     SELECT 'd' ||
6         COALESCE(LPAD( MAX(SUBSTR(numdep,2,5))+1,5,'0'),
7                     '00001')
8     INTO :NEW.numdep FROM departements; -- Ici
9 END;
```

Ok si insertion d'une seule ligne
Erreur SI insertion de plusieurs lignes

Déclencheurs réagissant aux instructions du LDD

La syntaxe

```
1 CREATE TRIGGER nom_déclencheur
2 BEFORE | AFTER
3 instruction_DDL ON Schema | DATABASE
4 [bloc PL/SQL];
1 CREATE TRIGGER auditCreation
2 AFTER CREATE
3 ON SCHEMA
4 BEGIN
5 INSERT INTO log
6 VALUES ('Objet créé le : ' || CURRENT_DATE);
7 END;
```

Déclencheurs réagissant aux événements système

La syntaxe

```
1 CREATE TRIGGER nom_déclencheur
2 BEFORE | AFTER
3 événement_système ON Schema | DATABASE
4 [bloc PL/SQL];
1 CREATE OR REPLACE TRIGGER tlogoff
2 BEFORE LOGOFF
3 ON SCHEMA
4 BEGIN
5 INSERT INTO loginout
6 VALUES (DBMS_STANDARD.SYSEVENT ||
7 ' de ' || USER || ' le ' || CURRENT_DATE);
8 END;
```